

Building your own FLStock and FLIndices

a4a Training Course — Module 1.5

Danai Mantopoulou Palouka & Alessandro Ligas

2026-09-13

Where we are

You've already build from CSV files:

- an **FLStock** — 14 slots filled, 3 left for the model
- an **FLIndex** — the survey, with its timing

 This afternoon

The same thing, with **your** data. Then the first assessment.

The recipe

```
CSV -> data.frame -> FLQuant -> FLStock -> checks -> sca()  
                                \-> FLIndex -> FLIndices -/
```

Nothing new. The work is in the **details of your files**, not in FLR.

Your files will not look like ours

Our demo data is **long** — one row per age and year:

```
age, year, data  
1, 1998, 91485.1
```

Most people's data is **wide** — ages down, years across:

```
age, 1998, 1999, 2000, ...  
1, 91485.1, 90945.4, ...
```

Both are fine. Wide needs one extra step:

```
dat <- read.csv("catch_n.csv", row.names = 1)  
flq <- FLQuant(as.matrix(dat), dimnames = list(age = ages, year = yrs))
```

Decisions only you can make

Decision	Question
Plusgroup	Which age absorbs everything older?
fbar range	Which ages average into reported $\bar{F}$?
Year range	Where do you trust the catch data from?
Units	Numbers in thousands or units? Weights in kg or g?



These are assessment choices

FLR will not question them. Write down what you chose and why — you will be asked on Friday.

Useful when reshaping

```
setPlusGroup(stk, 6)           # collapse ages 6+ into a plusgroup  
window(stk, start = 2000)      # cut years, or extend with NA  
trim(flq, age = 1:6)          # subset a dimension by name  
standardUnits(stk)            # the units FLR expects, slot by slot
```

`setPlusGroup()` sums numbers and takes weighted mean weights — do not do it by hand.

When something is missing

You don't have	Do this
Discards	<code>landings = catch</code> , leave discard slots empty
<code>stock.wt</code>	Use <code>catch.wt</code> as an approximation, and say so
Age-varying <code>M</code>	One value everywhere: <code>FLQuant(0.2, ...)</code>
Maturity by year	One ogive, recycled across years

Check before you fit

```
validObject(stk)    # does it satisfy the class?  
verify(stk)         # negative weights? maturity > 1? unit mismatches?
```

 verify() will always flag three things

anyna, harvest and cohorts fail on **any** unfitted stock — stock, stock.n and harvest are empty on purpose. Ignore those three; read everything else.

Then spot-check **one value** per file. A transposed matrix gives a plausible, completely wrong assessment.

Then, the first fit

```
idxs <- FLIndices(MEDITS = idx)
```

```
fit <- sca(stock = stk, indices = idxs)
```

```
stk.fit <- stk + fit           # the estimated slots, filled in
```

```
plot(FLStocks(input = stk, assessment = stk.fit))
```

stk + fit puts stock.n, stock and harvest back into the object.

Summary

- The pipeline is always the same; only your files differ
- **Wide or long** — both work, wide needs `as.matrix()`
- Plusgroup, fbar range, year range and units are **your** decisions
- Check that you have filled all necessary slots
- `stk + fit` returns the assessed stock - fills the 3 missing ones

Next: **Module 1.6 — inspecting and visualising what you have built.**

Exercise

1. Run `handson_M1.5_build_from_data.R` — the demo stock, all the way to a first `sca()` fit
2. Then open `exercise_M1.5_own_data.R` and work through it with **your own files**
3. The script ends with a **PASS/FAIL checklist**. Get it all green before you fit - or be sure why they fail.